Clase 5 · Modelo de programación ARM e instrucciones en ensamblador

Organización y Arquitectura de Computadoras · M.I. José Antonio de Jesús Arredondo Garza · lunes 14 sep 2026 · Tema 3 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Registros de la Pico 1 (Cortex-M0)

Registros de propósito general de 32 bits:

Registro	Uso
R0–R7	Bajos: uso libre
R8–R12	Altos: acceso más restringido desde Thumb
R13 / SP	Puntero de pila: **no** guardar datos ahí
R14 / LR	Dirección de retorno de una subrutina
R15 / PC	Dirección de la instrucción en ejecución
PSR	Estado: banderas que usan los saltos condicionales

2. Registros de punto flotante de la Pico 2

S0–S31, 32 registros IEEE 754 de precisión simple (32 bits), exclusivos de la **FPU**; solo los operan instrucciones de la FPU (no ADD/SUB enteros).

> **Incompatibilidad (examen)**
>
> Código que usa la FPU **no corre en la Pico 1**: no tiene unidad de punto flotante.

3. Sintaxis de Inline Assembly en MicroPython

Con la directiva **thumb**, cada instrucción va **con paréntesis**, como función:

Ensamblador puro	Inline de MicroPython
`ADD RD, RN, RM`	`add(RD, RN, RM)` o `add(RD, RM)` (RD = RD + RM)

4. Movimiento y memoria

- `mov(RD, RM)`: copia; inmediato limitado a **8 bits** en inline.
- `ldr(RD, [RN, offset])`: carga desde memoria.
- `str(RD, [RN, offset])`: guarda en memoria.

5. Aritmética y lógica

- `add(RD, RN, RM)`, `sub(RD, RN, RM)`; `mul(RD, RN)` solo con dos operandos.
- Lógicas: AND, ORR (OR), EOR (XOR).

> **Sin división entera**
>
> El Cortex-M0 **no tiene división entera** en hardware: es RISC de bajo consumo y el divisor es caro en silicio y ciclos. Se hace por software (corrimientos y restas).

6. Corrimientos y saltos

- `lsl(RD, RN, shift)` y `lsr(RD, RN, shift)`: máximo **31 bits**; más da error al ensamblar.
- Saltos condicionales sobre las banderas: **BEQ** (igual), **BNE** (distinto), **BCC** (sin acarreo), **BMI** (negativo), **BVS** (overflow).
- Salto en Pico 2: **2 ciclos** (8–16 ns a 150 MHz); Pico 1: **3 ciclos**.

7. Instrucciones de la FPU (solo Pico 2)

Instrucción	Operación
`vadd`, `vsub`, `vmul`, `vdiv` (SD, SN, SM)	Aritmética; la división sí existe, es la más costosa
`vsqrt(SD, SN)`	Raíz cuadrada en hardware
`vmov`	Convierte/mueve entre registros enteros y flotantes
`vldr`, `vstr`	Carga/almacena entre la FPU y variables de MicroPython

"No esperen encontrar ni seno ni coseno": funciones trascendentales siguen por software.

Contexto y pendiente

- El formato de corrimientos con offset viene de los DSP; la MAC de la Pico 2 aún no es fácil de usar desde inline.
- Miércoles 16 sep sin clase (Independencia). Próxima: programas completos en las tarjetas. Manual *RP2040 Assembly Language Programming* pendiente de subir.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
Registros Pico 1	R0–R7 bajos, R8–R12 altos, SP, LR, PC, PSR
FPU Pico 2	S0–S31, IEEE 754 simple; incompatible con Pico 1
Inline	Paréntesis: `add(RD, RN, RM)`
mov inmediato	8 bits
División	No hay en Cortex-M0 (RISC); sí `vdiv` en la FPU
lsl/lsr	Máximo 31 bits
Saltos	BEQ, BNE, BCC, BMI, BVS; 2 ciclos (Pico 2) vs. 3 (Pico 1)

FPU

vadd, vsub, vmul, vdiv, vsqrt, vmov, vldr, vstr. Sin seno ni coseno.